DATA STRUCTURES AND ALGORITHMS

23CSE203 LAB

MANUAL

SUBMITTED BY	
NAME	GUMMADI VISHNU
ROLL NUMBER	AV.SC.U4CSE25113
YEAR/SEM/SECTION	2ST YEAR /SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR(SL GRADE)
DEPARTMENT	COMPUTING

INDEX

4			
5			
6			
7			

WEEK – 1

1.AIM:

A rocket launch system displays a count down before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a count down from n to 1, followed by launch.

INPUT CODE:

1.1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/1.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def countdown(n):
    if n <= 0:
        print("Launch")
        return
    print(n)
    countdown(n - 1)
n = int(input("Enter n: "))
countdown(n)
```

OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/1.1.py =
Enter n: 10
10
9
8
7
6
5
4
3
2
1
Launch
>>>
```

EXPLANATION:

1.def countdown(n): — this snippet creates the recursive function and takes the starting number n as input.

2.if n <= 0: — this snippet is the base case. When the count has gone below 1, we stop going deeper.

3.print("Launch") — this snippet prints the final launch message once the countdown is over.

4.return — this snippet ends the current function call so recursion does not continue forever.

5.print(n) — this snippet shows the current number on the screen (5, then 4, then 3, and so on).

6.countdown(n - 1) — this snippet is the recursive call. It asks the same function to continue from the next smaller number.

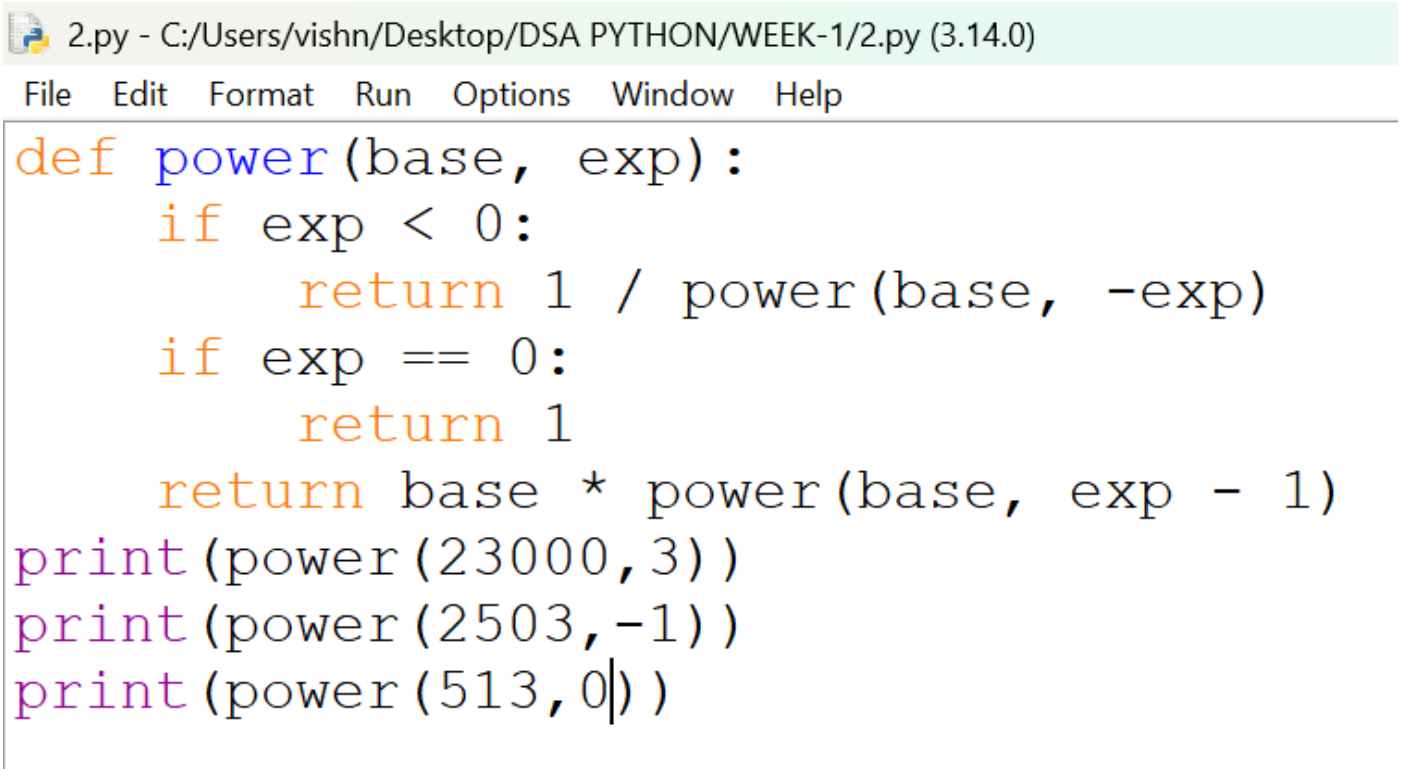
ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Base case if n <= 0 removed	RecursionError	maximum recursion depth exceeded	**Kee**p Launch + return

2.AIM:

Banks calculate the growth of an investment using compound interest. The formula requires calculating powers which can be implemented recursively. Write a recursive program to calculate P^n[power].

INPUT:



OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/2.py ====
121670000000000
0.00039952057530962844
1
>>>
```

EXPLANATION:

1.def power(base, exp): — this snippet defines the function that multiplies base by itself exp times.

2.if exp < 0: — this snippet handles a negative power (we need the reciprocal).

3.return 1 / power(base, -exp) — this snippet flips the exponent and divides 1 by that result (2^-3 → 0.125).

4.if exp == 0: — this snippet is the main base case. Any number to the power 0 is 1.

5.return 1 — this snippet stops recursion and sends 1 back to the previous call.

6.return base * power(base, exp - 1) — this snippet is the recursive step. It multiplies once and reduces the exponent by 1.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
if exp == 0: return 1 deleted	RecursionError	maximum recursion depth exceeded	**Keep power 0 = 1**
0 raised to a negative power	ZeroDivisionError	division by zero	**Treat 0^(-n) as undefined**

3.AIM:

An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

INPUT CODE:

3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/3.py (3.14.0)

File Edit Format Run Options Window Help

```
def search_id(ids, target, index=0):
    if index >= len(ids):
        return -1
    if ids[index] == target:
        return index
    return search_id(ids, target, index + 1)
print(search_id([101, 205, 330, 412, 550, 678],330))
print(search_id([101, 205, 330, 412, 550, 678],990))
```

OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/3.py
2
-1
>>>
```

EXPLANATION:

1.def search_id(ids, target, index=0): — this snippet starts the search. ids is the list, target is the ID we want, and index begins at 0.

2.index=0 — this snippet sets the starting position so the first call always looks at the first employee.

3.if index >= len(ids): — this snippet is the "not found" base case. If we have walked past the last ID, the list is finished.

4.return -1 — this snippet tells the caller that the employee ID does not exist in the list.

5.if ids[index] == target: — this snippet checks whether the current employee ID matches the one we are looking for.

6.return index — this snippet is the "found" base case. It sends back the position where the ID was found.

7.return search_id(ids, target, index + 1) — this snippet is the recursive call. It moves one step forward in the list and searches again.

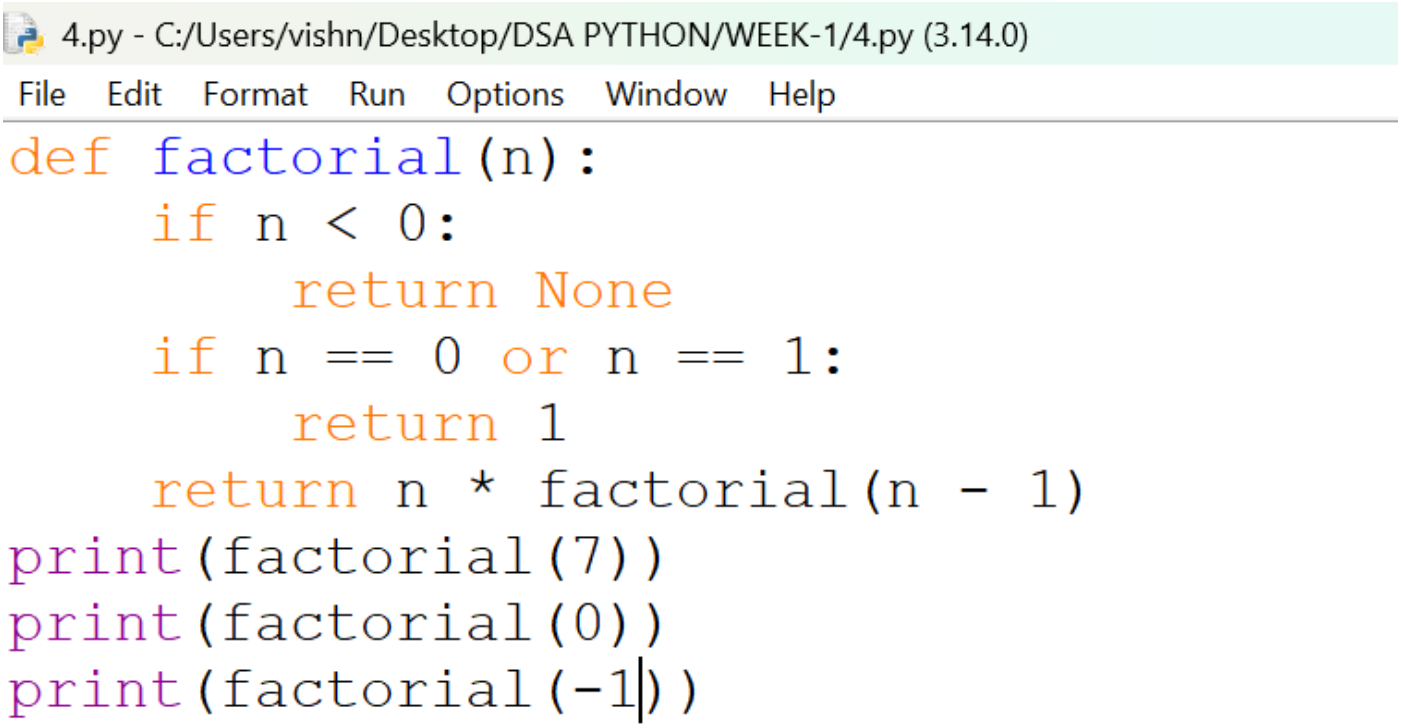
ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Input Ravi instead of a number	ValueError	invalid literal for int()...	**IDs are digits only**
if index >= len(ids) missing	IndexError	list index out of range	**Stop and return -1**

4.AIM:

Write a python program to find n! -factorial of a number which can be used to find the number of possible ways to arrange in different parcels.

INPUT CODE:



OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/4.py ==
5040
1
None
>>>
```

EXPLANATION:

1.def factorial(n): — this snippet defines the function that finds n! (ways to arrange parcels).

If n < 0: — this snippet catches invalid input; factorial is not defined for negatives.

return None — this snippet signals “no answer” so the program can print an error.

if n == 0 or n == 1: — this snippet is the base case. Both 0! and 1! are 1.

return 1 — this snippet stops recursion and starts sending values back up.

return n * factorial(n - 1) — this snippet is the recursive step. It multiplies the current number by the factorial of the previous one.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Input -4	Logical / handled	Factorial is not defined for negative numbers	**if n < 0: return None**
Base case 0! / 1! removed	RecursionError	maximum recursion depth exceeded	**Keep return 1**
Input 4.5	ValueError	invalid literal for int()...	**Enter a whole number**

5.AIM:

Write a python program to print first n terms of a Fibonacci series.

INPUT CODE:

5.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/5.py (3.14.0)

File Edit Format Run Options Window Help

```
def fib(k):
    if k <= 0:
        return 0
    if k == 1:
        return 1
    return fib(k - 1) + fib(k - 2)

def print_fibonacci(n, i=0):
    if n <= 0:
        print("No terms to display")
        return
    if i == n:
        print()
        return
    print(fib(i), end=" ")
    print_fibonacci(n, i + 1)
print(print_fibonacci(7))
print(print_fibonacci(2))
print(print_fibonacci(-5))
```

OUTPUT CASES:

```
>>>
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/5.py
0 1 1 2 3 5 8
None
0 1
None
No terms to display
None
>>>
```

EXPLANATION:

def fib(k): — this snippet finds one Fibonacci number at position k.

if k <= 0: return 0 — this snippet is the first base case. The 0th term is 0.

if k == 1: return 1 — this snippet is the second base case. The 1st term is 1.

return fib(k - 1) + fib(k - 2) — this snippet is the heart of Fibonacci. It adds the previous two terms.

def print_fibonacci(n, i=0): — this snippet prints the first n terms. i is which term we are on.

if n <= 0: — this snippet handles a bad request (0 or negative count).

print("No terms to display") — this snippet shows a clear message instead of an empty series.

print(fib(i), end=" ") — this snippet prints the current term on the same line with a space.

print_fibonacci(n, i + 1) — this snippet continues the series by moving to the next term.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
n = 0 or -5	Logical / handled	No terms to display	**if n <= 0 already handles it**
Base cases F(0), F(1) deleted	RecursionError	maximum recursion depth exceeded	**Keep 0 and 1 as stops**

WEEK – 2

1.AIM:

Write the Python program for linear search.

INPUT CODE:

1.py - C:\Users\vishn\Desktop\DSA PYTHON\WEEK-2\1.py (3.14.0)

File Edit Format Run Options Window Help

```
def linear_search(arr, key):
    i = 0
    n = len(arr)
    while i < n:
        if arr[i] == key:
            return i
        i = i + 1
    return -1
arr = [45, 12, 78, 23, 56, 89, 10]
print("List:", arr)
key = int(input("Enter element to search: "))
pos = linear_search(arr, key)
if pos != -1:
    print("Element", key, "found at position", pos)
else:
    print("Element", key, "not found")
```

OUTPUT CASES:

```
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-2\1.py ======
List: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 12
Element 12 found at position 1
>>>
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-2\1.py ======
List: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 88
Element 88 not found
>>>
```

EXPLANATION:

def linear_search(arr, key): — this snippet creates the function. arr is the list and key is the number we want.

i = 0 — this snippet starts the scan at the first position.

n = len(arr) — this snippet stores how many elements are in the list.

while i < n: — this snippet keeps checking until we have looked at every element.

if arr[i] == key: — this snippet compares the current element with the search key.

return i — this snippet stops the search and sends back the position where the key was found.

i = i + 1 — this snippet moves one step to the right when the current element is not the key.

return -1 — this snippet means we finished the list and the key is not there.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
User types letters (abc)	Runtime — ValueError	ValueError: invalid literal for int() with base 10: 'abc'	**Enter only numbers**
Loop condition written as while i <= n	Runtime — IndexError	IndexError: list index out of range	**Use while i < n**
Searching a value that is not in the list (99)	Logical (no crash)	Element 99 not found	**Already handled by return -1**

2.AIM:

2.Write the python code for the binary search with:

a) Sorted list of numbers

INPUT CODE:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1


arr = [10, 20, 30, 40, 50, 60, 70]
print("Sorted list:", arr)
key = int(input("Enter element to search: "))
pos = binary_search(arr, key)
if pos != -1:
    print("Element", key, "found at position", pos)
else:
    print("Element", key, "not found")
```

OUTPUT CASES:

```
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.py ==
Sorted list: [10, 20, 30, 40, 50, 60, 70]
Enter element to search: 20
Element 20 found at position 1
>>>
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.py ==
Sorted list: [10, 20, 30, 40, 50, 60, 70]
Enter element to search: 80
Element 80 not found
>>>
```

EXPLANATION:

def binary_search(arr, key): — this snippet defines binary search. The list arr must already be sorted.

low = 0 — this snippet marks the left end of the current search range.

high = len(arr) - 1 — this snippet marks the right end of the current search range.

while low <= high: — this snippet continues as long as there is still a valid range to search.

mid = (low + high) // 2 — this snippet finds the middle index (integer division, no decimal).

if arr[mid] == key: — this snippet checks whether the middle element is the one we want.

return mid — this snippet is the success stop. The key is at mid.

elif arr[mid] < key: — this snippet means the key is bigger, so it must be on the right side.

low = mid + 1 — this snippet throws away the left half, including mid.

else: high = mid - 1 — this snippet means the key is smaller, so we throw away the right half.

return -1 — this snippet means the range became empty and the key is not in the list.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
User types text (forty)	Runtime — ValueError	ValueError: invalid literal for int()...	**Enter a number**
while low < high instead of low <= high	Logical	Key exists but program says **not found**	**Use while low <= high**
mid = (low + high) / 2 (single slash)	Runtime / logical	TypeError: list indices must be integers	**Use integer division //**

2.b.AIM:

b) Unsorted list of numbers.

Write the Python program for linear search.

INPUT CODE:

2.2 un sorted.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.2 un sorted.py (3.14.0)

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1


def binary_search_unsorted(arr, key):
    sorted_arr = sorted(arr)          # sort first
    pos = binary_search(sorted_arr, key)
    return sorted_arr, pos


arr = [45, 12, 78, 23, 56, 89, 10]
print("Original (unsorted) list:", arr)
key = int(input("Enter element to search: "))
sorted_arr, pos = binary_search_unsorted(arr, key)
print("After sorting:", sorted_arr)
if pos != -1:
    print("Element", key, "found at position", pos, "in the sorted list")
else:
    print("Element", key, "not found")
```

OUTPUT CASES:

```
====== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.2 un sorted.py ==
Original (unsorted) list: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 78
After sorting: [10, 12, 23, 45, 56, 78, 89]
Element 78 found at position 5 in the sorted list
>>>
====== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.2 un sorted.py ==
Original (unsorted) list: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 99
After sorting: [10, 12, 23, 45, 56, 78, 89]
Element 99 not found
```

EXPLANATION:

sorted_arr = sorted(arr) — this snippet makes an ascending copy. Binary search is safe only after this.

pos = binary_search(sorted_arr, key) — this snippet runs the same binary search on the sorted copy.

return sorted_arr, pos — this snippet sends back both the sorted list (so we can print it) and the index.

print("After sorting:", sorted_arr) — this snippet shows that we sorted before searching.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Empty list []	Logical (no crash)	not found	**Correct for an empty list**

WEEK – 3

1.AIM:

Write the Python program to implement bubble sort where the elements are taken from the user.

INPUT CODE:

1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/1.py (3.14.0)

File Edit Format Run Options Window Help

```
def bubble_sort(arr):
    n = len(arr)
    i = 0
    while i < n - 1:
        j = 0
        swapped = False
        while j < n - i - 1:
            if arr[j] > arr[j + 1]:
                temp = arr[j]
                arr[j] = arr[j + 1]
                arr[j + 1] = temp
                swapped = True
            j = j + 1
        if not swapped:
            break
        i = i + 1
    return arr


n = int(input("Enter number of elements: "))
print("Enter", n, "elements (space separated):")
arr = list(map(int, input().split()))
print("Before sorting:", arr)
print("After bubble sort:", bubble_sort(arr))
```

OUTPUT CASES:

```
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-3\1.py ==========
Enter number of elements: 4
Enter 4 elements (space separated):
3 5 7 9
Before sorting: [3, 5, 7, 9]
After bubble sort: [3, 5, 7, 9]
>>>
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-3\1.py ==========
Enter number of elements: 5
Enter 5 elements (space separated):
23 34 632 2 1
Before sorting: [23, 34, 632, 2, 1]
After bubble sort: [1, 2, 23, 34, 632]
>>>
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-3\1.py ==========
Enter number of elements: 5
Enter 5 elements (space separated):
22 1 54 43
Before sorting: [22, 1, 54, 43]
After bubble sort: [1, 22, 43, 54]
>>>
```

EXPLANATION:

n = int(input("Enter number of elements: ")) — this snippet asks how many numbers the user wants to sort.

arr = list(map(int, input().split())) — this snippet reads the numbers from one line and turns them into a list of integers.

i = 0 / while i < n - 1: — this snippet is the outer pass. After pass i, the last i elements are already in place.

j = 0 / while j < n - i - 1: — this snippet walks through the unsorted part, comparing neighbours.

if arr[j] > arr[j + 1]: — this snippet checks whether two neighbours are in the wrong order.

temp = arr[j] … arr[j + 1] = temp — this snippet swaps the two neighbours.

swapped = True — this snippet remembers that at least one swap happened in this pass.

if not swapped: break — this snippet stops early if the list is already sorted (no swaps).

return arr — this snippet sends the sorted list back to be printed.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
n typed as text (five)	Runtime — ValueError	ValueError: invalid literal for int() with base 10: 'five'	**Enter a whole number for n**
Inner loop written as while j < n	Runtime — IndexError	IndexError: list index out of range	**Use while j < n - i - 1**

n = 0 or n = -4	Logical / handled	Number of elements must be a positive integer	**Enter n ≥ 1**

2.AIM:

Write the Python program to implement insertion sort where the elements are taken from the user.

INPUT CODE:

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/2.py (3.14.0)

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    n = len(arr)
    i = 1
    while i < n:
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1
        arr[j + 1] = key
        i = i + 1
    return arr


n = int(input("Enter number of elements: "))
print("Enter", n, "elements (space separated):")
arr = list(map(int, input().split()))
print("Before sorting:", arr)
print("After insertion sort:", insertion_sort(arr))
```

OUTPUT CASES:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/2.py ============
Enter number of elements: 6
Enter 6 elements (space separated):
23 535 62 4 54 234
Before sorting: [23, 535, 62, 4, 54, 234]
After insertion sort: [4, 23, 54, 62, 234, 535]
>>>
```

EXPLANATION:

i = 1 / while i < n: — this snippet starts from the second element. The first element is already a sorted list of size 1.

key = arr[i] — this snippet picks the next value that must be inserted into the left (sorted) part.

j = i - 1 — this snippet starts comparing key with the element just to its left.

while j >= 0 and arr[j] > key: — this snippet shifts larger elements one step right until the hole for key is found.

arr[j + 1] = arr[j] — this snippet moves a bigger element to the right to make space.

j = j - 1 — this snippet walks further left.

arr[j + 1] = key — this snippet drops key into the correct place.

i = i + 1 — this snippet moves on to the next unsorted element.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
A list value is not a number (1 2 a 4)	Runtime — ValueError	ValueError: invalid literal for int() with base 10: 'a'	**Enter only integers**
Inner condition is only arr[j] > key (no j >= 0)	Runtime — IndexError	IndexError: list index out of range	**Keep while j >= 0 and arr[j] > key**
Outer loop starts at i = 0 instead of i = 1	Logical	----------	**Start from the second element (i = 1)**

3.AIM:

Write the Python program to implement selection sort where the elements are taken from the user.

INPUT CODE:

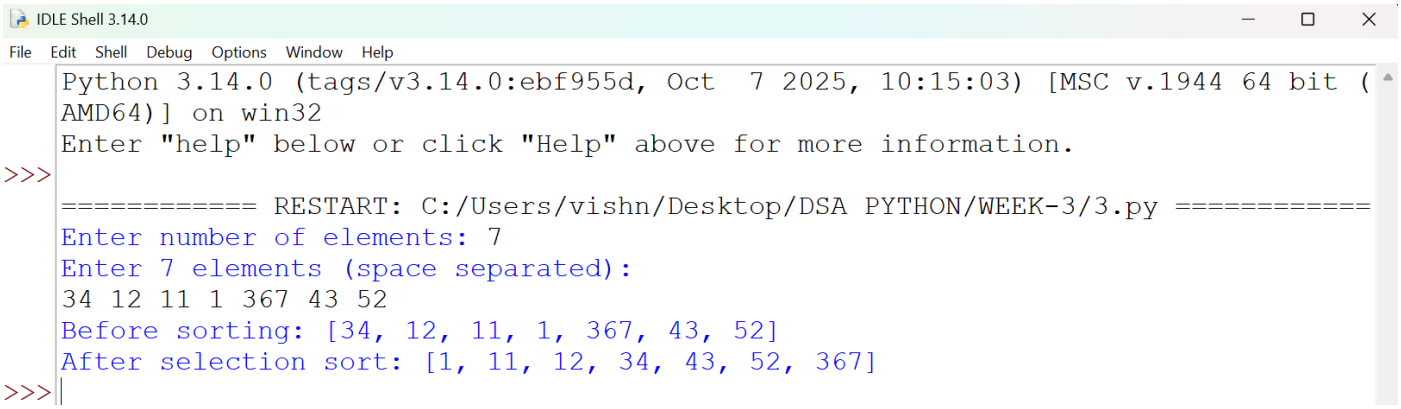
3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/3.py (3.14.0)

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n = len(arr)
    i = 0
    while i < n - 1:
        min_index = i
        j = i + 1
        while j < n:
            if arr[j] < arr[min_index]:
                min_index = j
            j = j + 1
        temp = arr[i]
        arr[i] = arr[min_index]
        arr[min_index] = temp
        i = i + 1
    return arr


n = int(input("Enter number of elements: "))
print("Enter", n, "elements (space separated):")
arr = list(map(int, input().split()))
print("Before sorting:", arr)
print("After selection sort:", selection_sort(arr))
```

OUTPUT CASES:



EXPLANATION:

i = 0 / while i < n - 1: — this snippet marks the position that should receive the next smallest value.

min_index = i — this snippet assumes the current position holds the smallest so far.

j = i + 1 / while j < n: — this snippet scans the rest of the list to find anything smaller.

if arr[j] < arr[min_index]: — this snippet updates the record when a smaller value is found.

min_index = j — this snippet remembers where that smaller value lives.

temp = arr[i] … arr[min_index] = temp — this snippet swaps the smallest remaining value into position i.

i = i + 1 — this snippet locks that position and continues with the rest of the list.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Too few numbers entered (n=4 but only 10 20)	Logical / handled	You entered 2 values, expected 4	**Type exactly n numbers**
Forgot to reset min_index = i each pass	Logical	List is not fully sorted	**Set min_index = i at the start of every outer pass**
Compared with > instead of <	Logical	List comes out in descending order	**Use if arr[j] < arr[min_index] for ascending**